



The Hashemite University

Faculty of Prince Al-Hussein Bin Abdallah II for Information Technology

Information Technology Department

Arabic Handwriting Recognition

A Project submitted

**in partial fulfillment of the requirement for the
B.Sc. Degree in Data science and Artificial Intelligence**

By

Sada Mohammad Abu Hasheesh (2136264)

Sondos Faraj Abu Mardyah (2230116)

Logain Jamal Fahmi Saleem (2144570)

Zaidoon Nidal Salameh Hijazeen (2137630)

Supervised by

Dr. Esra'a Ahmad Helael alshdaifat

Committee Member Names

Dr. Majdi Ahmad Mosa Maabreh

January 2025

CERTIFICATE

It is hereby certified that the project titled *Arabic Handwriting Recognition*, submitted by undersigned, in partial fulfillment of the award of the degree of “Bachelor in Data science and Artificial Intelligence” embodies original work done by them under my supervision. All the analysis, design and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university

Sada (2136264)

Signature

Sondos (2230116)

Signature

Logain (2144570)

Signature

Zaidoon (2137630)

Signature

ABSTRACT

Arabic handwritten character recognition presents a significant challenge due to the cursive nature of the Arabic script, where character forms change based on their position within a word. Variations in handwriting styles, the presence of diacritical marks, and overlapping characters further complicate the recognition process.

This project employs natural language processing, machine learning, and deep learning techniques to recognize Arabic handwritten characters. The system is trained on a diverse dataset of Arabic handwritten characters, enabling it to generalize effectively and accurately recognize characters across various handwriting styles.

The project aims to enhance the accuracy of Arabic character recognition, supporting applications such as document digitization, historical manuscript preservation, and assistive technologies for visually impaired individuals. By addressing the inherent challenges of the Arabic script, this system offers a robust and efficient solution for handwritten text recognition.

ACKNOWLEDGEMENTS

First and foremost, we sincerely thank Allah for granting us the remarkable opportunity to work on this project. We also extend our heartfelt gratitude to our families, whose unwavering support and encouragement have been the cornerstone of our success. Our deepest appreciation goes to our professors, whose patience and dedication have enriched our learning journey throughout our college years. Their commitment to our education has been instrumental in bringing us to this milestone. Lastly, we would like to offer a special thanks to our supervisor, Dr. Esra'a alshdaifat, whose guidance and mentorship in our project have profoundly shaped our journey in studying Artificial Intelligence. Her expertise and insights have not only directed us but also inspired us to pursue excellence in our field.

LIST OF FIGURES

Figure1: Google Keep.....	4
Figure 2: OCRSpace.....	5
Figure3: i2OCR	6
Figure 4 : AlexU-Word Example.....	7
Figure 5 : Arabic Character Recognition Example	7
Figure 6 : KHATT Example.....	8
Figure 7: HMBD-v1 Dataset.....	8
Figure8: A general overview of the proposed system.	10
Figure9:an example taken from the Synthetic Arabic Data for Scene Text Recognition dataset.....	12
Figure 10: Image Preprocessing: Before and After Applying Techniques (Resize, Normalize, Denoise, Grayscale)	14
Figure 11: Image Preprocessing After Applying Techniques (Resize, Normalize, Denoise, Grayscale, Enhance Contrast)	15
Figure 12: Image Preprocessing: After Applying Techniques (Resize, Normalize, Denoise, Grayscale, Augment Data).	17
Figure 13: Image Preprocessing: After Applying Techniques (Resize, Normalize, Denoise, Grayscale, Enhance Contrast, Augment Data)	17
Figure 14 – High-level system architecture of the Arabic OCR pipeline.....	22
Figure 15: Empty upload area indicating where the user can add image files.....	32
Figure 16: Highlighted drag-and-drop zone prompting file selection.....	32
Figure 17: Native file-open dialog showing available image files.....	33
Figure 18: “Start Analysis” button pressed to begin preprocessing and model inference.....	33
Figure 19: Result page displaying the uploaded handwriting sample and its correct prediction “طريق نور صعب”.....	34
Figure 20: Result page displaying a second handwriting sample and its correct prediction “سعيد ضرب نور” along with the option to upload another image.....	34
Figure 21: Example of input validation rejecting non-Arabic text such as English characters and numbers.....	35

LIST OF TABLES

Table 1: Project Schedule.....	3
Table 2:Accuracy Comparison Across Different Dataset Combinations.....	28
Table 3: SVM Model Performance Across Different Dataset Combinations.....	36
Table 4: Random Forest Model Performance Across Different Dataset Combinations...	37
Table 5:XGBoost Model Performance Across Different Dataset Combinations.....	37
Table 6 : CNN Model Performance Across Hyperparameter Configurations.....	38
Table 7 : Directory structure of the deployed.....	43

CHAPTER 1: INTRODUCTION

This chapter gives a clear explanation of the project. It also covers the project's main idea, problem statements, aims and objectives, and project limitations.

1.1 Overview

Arabic handwritten text recognition has become an essential technology in today's rapidly digitizing world. As governments, businesses, and educational institutions shift towards more automated processes, the ability to efficiently recognize and process handwritten text is critical. Arabic script, however, poses unique challenges compared to other languages due to its nature. More specifically, in Arabic text, the shapes of letters change depending on their position in the word. Additionally, the presence of diacritics is considered another challenge when handling Arabic text.

In recent years, advancements in deep learning, particularly Convolutional Neural Networks (CNNs), have shown promise in addressing these challenges. CNNs have proven highly effective at feature extraction and can handle the variability in handwriting styles and the complexity of Arabic script. This project aims to use these advancements and develop a robust system capable of accurately recognizing Arabic handwritten text. The system will contribute to various applications such as document digitization and archival, enabling more efficient and accurate handling of Arabic handwritten content.

1.2 Project Motivation

Arabic character recognition presents significant challenges due to the script's cursive nature, contextual letter forms, and diacritical marks. Despite the increasing need for Arabic text digitization in fields like education, accessibility, and historical preservation, existing solutions are limited and often inaccurate.

This project utilizes machine learning and deep learning algorithms to develop a robust Arabic character recognition system, addressing these challenges.

1.3 Problem Statement

Arabic handwriting recognition systems face significant challenges due to the complexity of the Arabic script, where letter forms change depending on their position in a word (beginning, middle, end, or isolated), along with the connections between letters in cursive writing. This complexity makes it difficult for systems to accurately recognize handwritten text. Additionally, the Arabic language includes numerous diacritics that affect the meaning of words, further complicating the task of correct interpretation.

The current existing systems for Arabic text recognition are associated with limited accuracy and robustness. This is largely due to the lack of comprehensive datasets that capture all possible variations of letter and word forms. This data gap restricts the ability of these systems to handle the wide variety of handwriting styles and diacritical variations, resulting in limited recognition accuracy.

1.4 Project Aim and Objectives

The primary goal of this project is to develop an effective model capable of accurately recognizing Arabic handwritten text. To reach this goal, our project aims to overcome the challenges associated with Arabic text, such as letter forms and diacritics, by utilizing a comprehensive dataset and machine learning and deep learning techniques to enhance recognition accuracy. The project will deliver an effective tool for document digitization and archival.

1.4.1 Data Collection and Preparation:

The project will utilize the **Synthetic Arabic Data for Scene Text Recognition** dataset, which consists of **50,000 cropped images** containing embedded Arabic text. The labels for these images were generated from an Arabic word's corpus containing **15,000 words**. Additionally, the dataset includes **100 cropped images** collected from Arabic hashtags on Twitter. These datasets were originally used in the published paper *"Detecting Spam Images with Embedded Arabic Text in Twitter."*

Before training the model, the dataset will undergo a preprocessing phase to ensure high data quality. Preprocessing steps will include:

1. **Converting images to grayscale:** This reduces complexity and focuses on essential features.
2. **Noise reduction:** To remove artifacts or distortions that could interfere with the recognition process.
3. **Resizing images:** Ensuring consistency in input dimensions for the training model.

This dataset provides a robust foundation for training and testing the Arabic handwritten text recognition system, as it captures a diverse range of Arabic text appearances.

1.5 project Schedule:

Table 1: Project Schedule

Duration	End Date	Start Date	Task
weeks 2	25-10-2024	10-10-2024	Project Kickoff and Goal Definition
weeks 3	15-11-2024	26-10-2024	Data Collection and Initial Exploration
weeks 3	06-12-2024	16-11-2024	Data Preprocessing and Feature Engineering
weeks 3	25-12-2024	07-12-2024	Model Development and Initial Training
weeks 2	10-01-2025	26-12-2024	Model Evaluation and Testing
weeks 2	24-01-2025	11-01-2025	Deployment and Web Integration
weeks 2	07-02-2025	25-01-2025	Final Report Writing and Review
week 1	15-02-2025	08-02-2025	Presentation and Project Delivery

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems

Several systems have been developed to address handwriting recognition, including OCR tools and custom solutions for Arabic text. Below is an overview of key systems, how they work, and how images can be used for text extraction:

- **Google Keep**

Google Keep is a multi-functional tool with OCR capabilities that can extract text from images. It supports a wide range of languages, including Arabic, and uses advanced machine learning algorithms to analyze and identify text. Users can upload an image directly to the application, and Google Keep processes the image to extract the embedded text. However, it primarily targets printed text and does not account for the cursive and contextual complexities of Arabic handwriting. Despite its ease of use, Google Keep lacks specialized features for Arabic handwriting recognition, such as diacritical handling or positional letter variations, as illustrated in *Figure 1*.

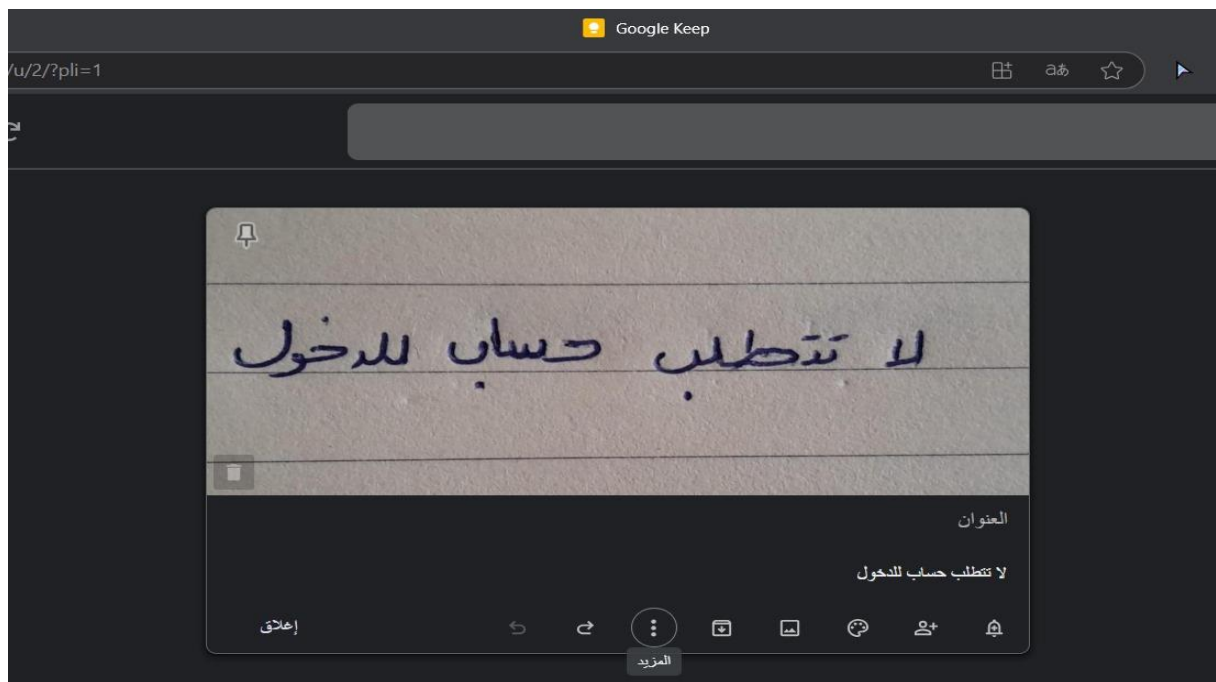


Figure 1: Google Keep

- **OCR Space**

OCR Space is an online OCR platform that supports text extraction for multiple languages, including Arabic. It processes uploaded images through pre-trained OCR models. The user uploads an image to the platform, and the system identifies the text within the image. While effective for structured and clear images, its accuracy diminishes significantly when dealing with noisy or handwritten text. OCR Space offers a simple interface, making it accessible for non-technical users, but it lacks the ability to handle the dynamic nature of Arabic handwriting (Figure 2) .

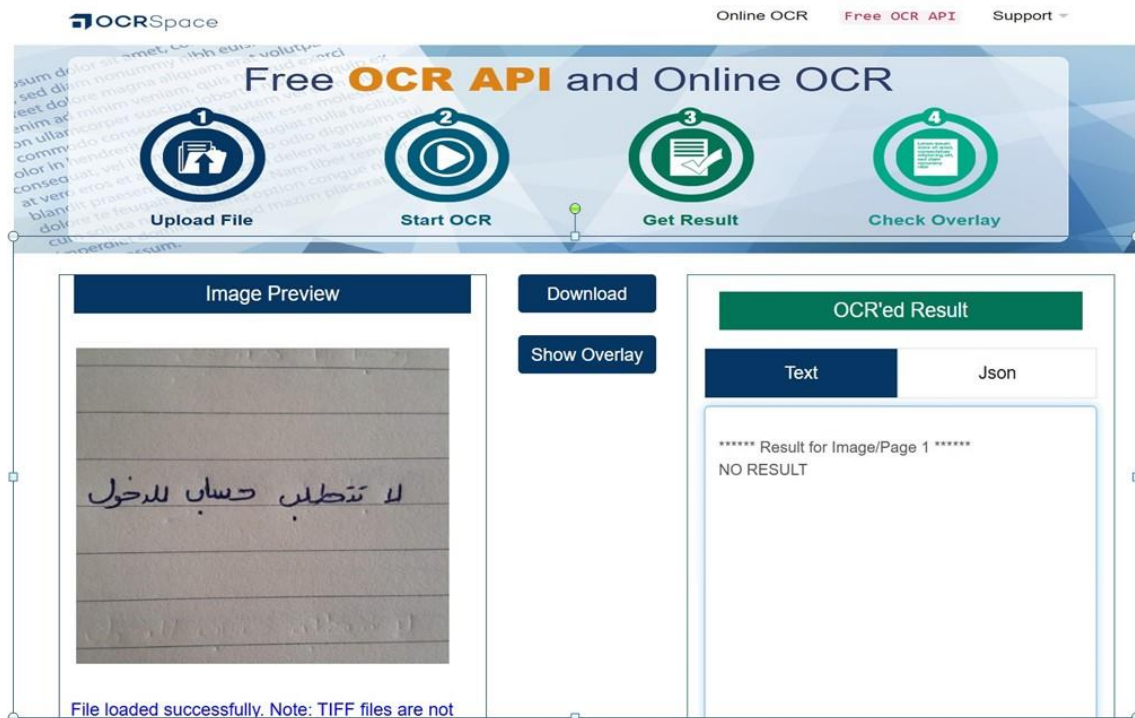


Figure 2: OCR Space

- **i2OCR**

i2OCR provides a free online OCR solution capable of recognizing Arabic text from images. The system is designed to convert images into editable text files. Users upload an image containing text, and i2OCR applies its pre-trained models to recognize and extract the text. While effective for printed text, it struggles with handwritten scripts due to insufficient contextual understanding. Additionally, the system does not include

preprocessing techniques to enhance text clarity, which limits its utility for handwriting recognition (Figer3).

Each of these systems offers general functionalities for text recognition but falls short in addressing the unique characteristics of Arabic handwriting, such as the inclusion of diacritical marks, variable letter forms, and cursive connections.

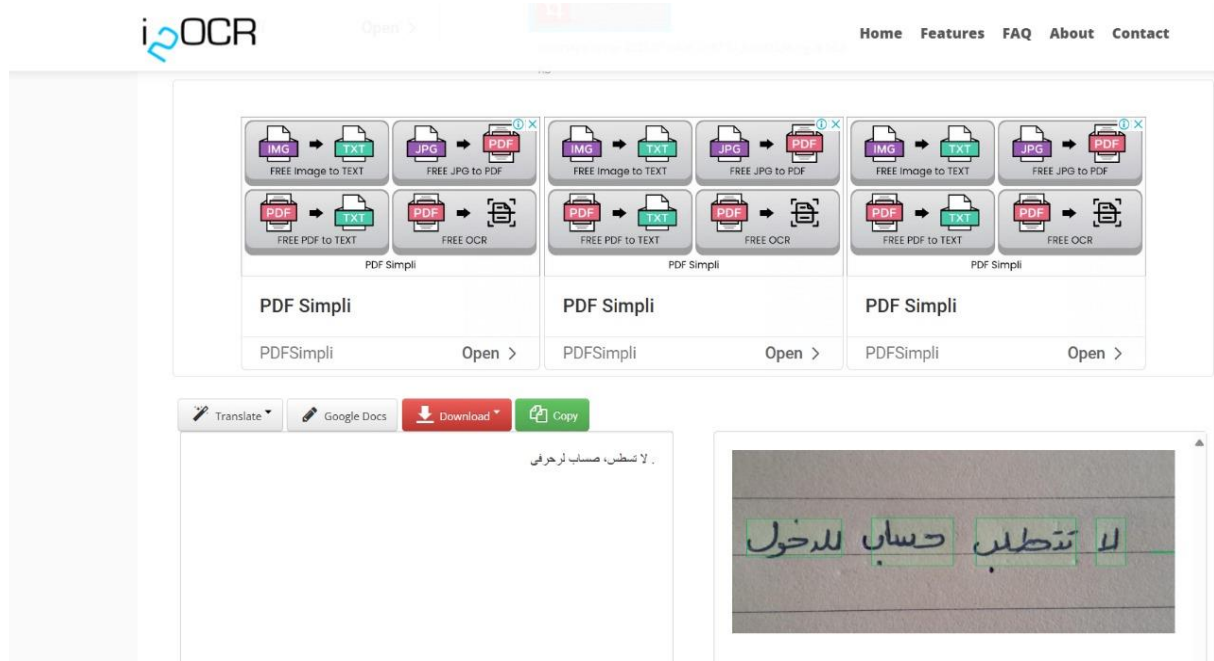


Figure 3: i2OCR

2.1.2 Existing Dataset:

The AlexU-Word dataset, Arabic Character Recognition dataset, and KHATT dataset collectively form a robust foundation for advancing Arabic handwriting recognition technologies.

1. AlexU-Word Dataset

The AlexU-Word dataset contains over 25,000 images of handwritten Arabic words collected from 907 writers, emphasizing diversity in handwriting styles. Each image represents an isolated word, and the dataset has undergone extensive preprocessing, including cropping and noise removal, to ensure high quality. Feature extraction techniques like HOG and SIFT have been applied, and the dataset is particularly useful for training models like k-

NN, SVM, and CNN. Figure 4 shows an example of this dataset, which is widely used in OCR systems and handwriting analysis research.



Figure 4 : AlexU-Word Example

2. Arabic Character Recognition Dataset

This dataset addresses challenges in character-level recognition, providing 16,800 images of handwritten Arabic characters. These are split into 13,440 training samples and 3,360 testing samples, collected from 60 individuals. The dataset has undergone preprocessing steps such as resizing, normalization, and denoising, resulting in high-resolution images ideal for CNNs. Figure 5 illustrates a sample from this dataset. This dataset is vital for automating document digitization and creating educational tools for Arabic language learners.



Figure 5 : Arabic Character Recognition Example

3. KHATT Dataset

The KHATT dataset offers a comprehensive resource for sequence-based Arabic handwriting recognition, containing 70,000 sequences of handwritten text resized to 64x800 pixels. Images are normalized, and labels are encoded into numeric sequences for uniformity. Data is split into 70% training, 15% validation, and 15% testing sets. The dataset supports temporal sequence modeling using BLSTM and is valuable for developing RNNs and automating tasks like document transcription and manuscript digitization. Models trained on

KHATT achieved 84% character-level accuracy and 71% word-level accuracy, showcasing its potential even without data augmentation. Figure 6 provides an example from this dataset.

A sample of handwritten Arabic text from the KHATT dataset. The text is written in a cursive style on a light blue grid background. The text reads: "واكتسبت كل الوسائط المتعددة، وبلغ عدد الطلاب في كل الدورات".

Figure 6 : KHATT Example

4.HMBD-v1 Dataset:

The is a modern collection of handwritten Arabic characters, offering 45,000 samples that represent all 28 Arabic letters in various handwriting styles. This dataset was created to enhance the development of deep learning models for Arabic character recognition. Data samples are divided into training, validation, and testing sets to support comprehensive model evaluation. The dataset's diversity in styles makes it particularly useful for improving model generalization and robustness. which is frequently utilized in educational tools, automated text analysis, and Arabic OCR systems. Figure 7 provides an example from this dataset.

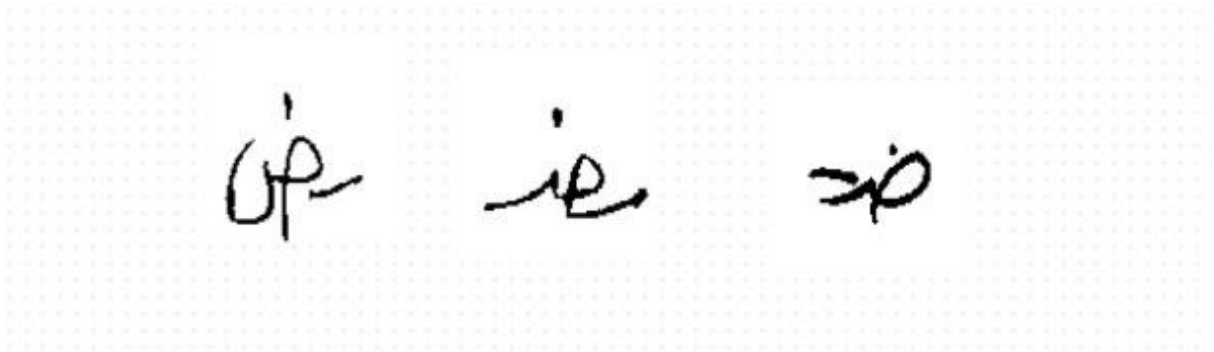


Figure 7: HMBD-v1 Dataset

2.2 Limitations of Existing Systems

Existing systems face several challenges:

- **Lack of Support for Diacritics and Connected Letters:** Many systems struggle to handle diacritical marks and connected letters, especially in cursive scripts like Arabic. These marks affect how words are pronounced and understood, and the shape of

letters changes depending on where they are in a word. This makes it harder for systems to recognize words correctly, especially in handwritten text.

Technological Gaps: Current technologies struggle to accommodate the cursive nature of Arabic script and the contextual changes in letter forms.

- **Generalization Issues:** Pre-trained models frequently fail to adapt to different handwriting styles, fonts, and word structures, reducing their accuracy and reliability.

2.3 Overall Solution Approach

To address the identified limitations, this project proposes a comprehensive solution using advanced data processing and modeling techniques, specifically designed for Arabic language text recognition. The approach includes specialized methods and models to improve accuracy and handle the unique challenges of Arabic script.

- **Synthetic and Augmented Datasets:** Employing synthetic datasets to simulate a wide range of handwriting styles and augmenting them with real-world samples for enhanced diversity.
- **Deep Learning Models:** use CNNs and transformer-based architectures like BERT for effective feature extraction and contextual analysis.
- **Advanced Preprocessing:** Utilizing techniques such as noise reduction, normalization, and data augmentation to prepare high-quality inputs for the model.
- **User-Friendly Design:** Developing a user interface that facilitates efficient interaction with the recognition system, enabling practical applications in document digitization and archival

CHAPTER 3: REQUIREMENT ENGINEERING AND ANALYSIS

This chapter establishes the foundation for the development of a robust Arabic handwriting recognition system. It details the data collection process, preprocessing, and feature engineering strategies while addressing ethical considerations to ensure responsible implementation. These efforts ensure the project achieves high accuracy and scalability, enabling its application in various domains such as document digitization and historical preservation. Figur7 presents a general overview of the proposed system. For generating the desired system, the following phases will be followed:

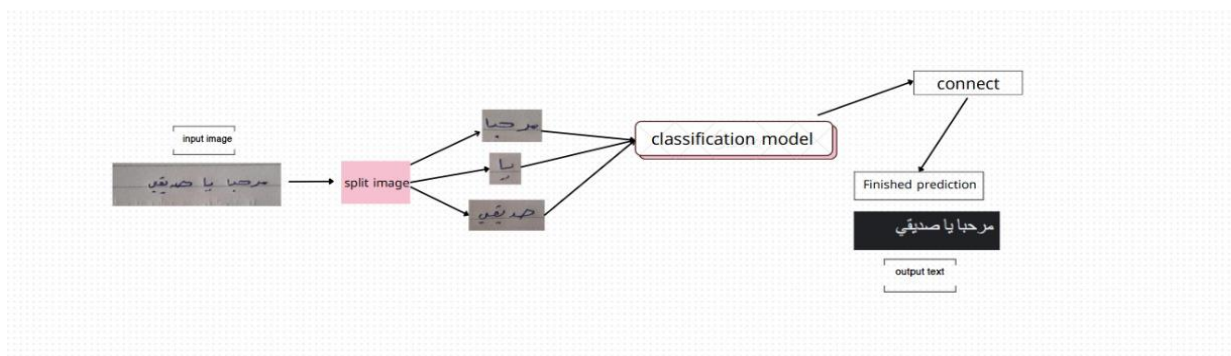


Figure 8: A general overview of the proposed system.

1. Input Image:

- The process starts with an input image containing text (e.g., "مرحباً يا صديقي").

2. Split Image:

- The image is divided into smaller segments, specifically based on words (e.g., "مرحباً", "يا", "صديقي").

3. Word Split:

- Each word is processed separately, meaning the system isolates individual words from the image.

4. Model System:

- Each word segment is sent to a pre-trained model for predicting the word.

5. Reassembling (Connect):

- After recognizing each word, the system combines them in the correct order to reconstruct the full text.

6. Output Text:

- The final text (e.g., "مرحباً يا صديقي") is generated after the model has processed and predicted all words.

3.1 Data Understanding and Preparation:

The data understanding and preparation phase is one of the most critical stages in building a robust Arabic handwriting recognition system. The quality, diversity, and preprocessing of the data directly impact the model's accuracy, generalizability, and overall performance. This section explains the detailed process of data collection, preprocessing, and integration to prepare the dataset for training.

3.1.1 Data Collection Methods and Sources:

Synthetic Arabic Data for Scene Text Recognition Dataset that taken from <https://bit.ly/3BSGOAe> is utilized. The main characteristics of the dataset could be summarized as follows:

- This dataset contains 50,000 images of Arabic words, generated synthetically.
- The text is derived from a corpus of 15,000 Arabic words, ensuring the dataset covers a wide range of vocabulary.
- The synthetic nature allows the inclusion of controlled variations in text, such as:
- Different Fonts: Both traditional and modern Arabic fonts are represented to simulate printed and handwritten text.
- Diacritical Marks: The dataset includes words with and without diacritics (e.g., "سلام" vs. "سَلام") to handle variations in Arabic spelling.
- Word Positions: Variability in word alignment, such as centered, left-aligned, or slightly tilted text.

Why Synthetic Data is Useful

Synthetic data allows us to generate large-scale, clean datasets with predefined attributes, making it an effective solution for overcoming the scarcity of real-world handwritten Arabic datasets, which are often noisy and difficult to obtain. By using synthetic data, we can create diverse and high-quality samples that improve model training and generalization. This is illustrated in *Figure 8*, which shows an example taken from the Synthetic Arabic Data for Scene Text Recognition dataset.

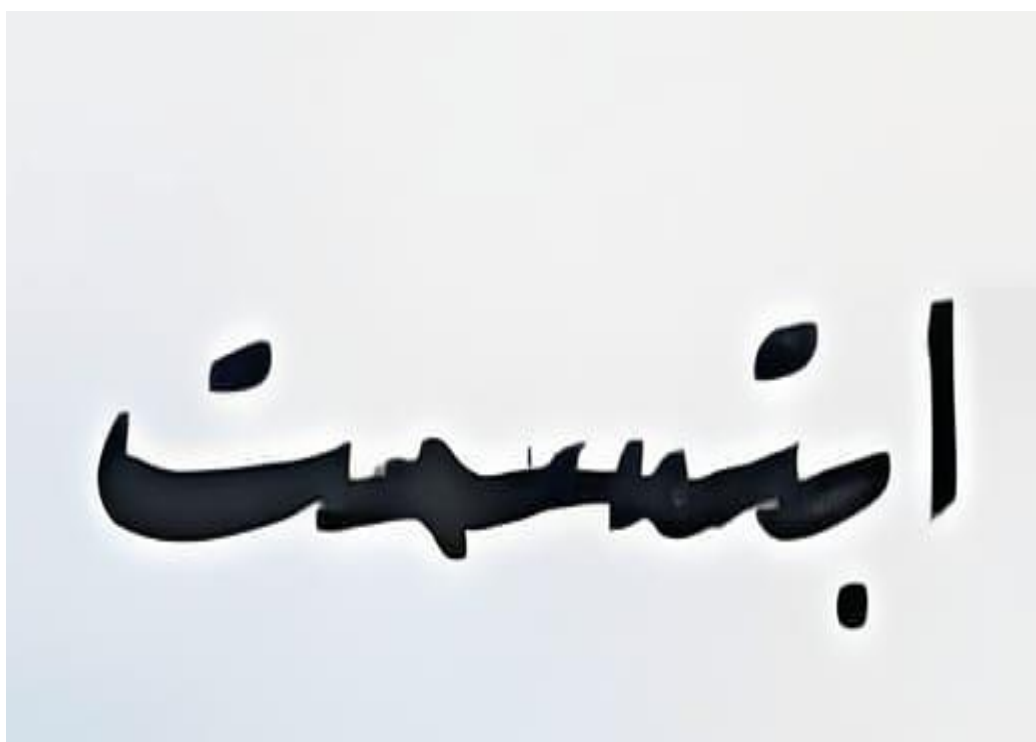


Figure 9 an example taken from the Synthetic Arabic Data for Scene Text Recognition dataset.

3.1.2 Data Preprocessing Techniques and Cleaning.

3.2 Data Cleaning:

The following steps are conducted to clean the used dataset:

Removing Duplicate Images:

Method: The `processed_images` set was used to track the base names of images and prevent processing of images with duplicate names. The full image path was used to

determine if the image had been processed before. The base name (before the _ in the path) was extracted and used to track duplicates. If an image with the same base name was found, it was skipped and not processed.

Excluding Unreadable Images:

Method: Images that cannot be read or contain issues were logged in an error log file (error_log_path) and removed. This was achieved using a try-except block to catch errors during image loading or processing.

Removing Invalid Images:

Method: In this phase, low-quality images were excluded from the dataset. We used noise detection and contrast checks to identify low-quality images. Specifically, the Pillow library was used to convert images to RGB format, calculate the contrast using ImageStat.Stat, and compute the standard deviation using numpy.

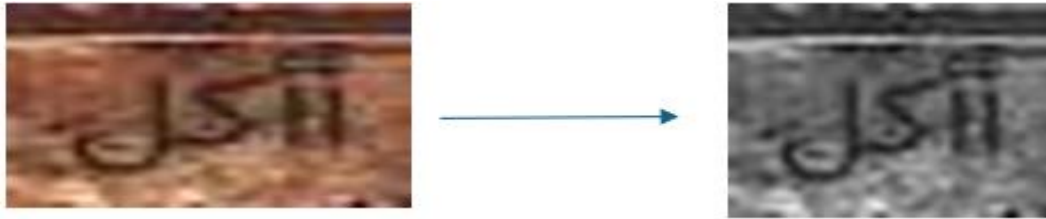
We tried several pre-processing techniques to achieve a high-performance prediction model. The initial processing steps include:

First Processing Step:

In the initial preprocessing phase, several techniques were applied to prepare the images for model training. First, images were **resized** to match the required dimensions, reducing complexity and ensuring compatibility with the model's architecture. Next, **normalization** was performed to adjust pixel values to a standard range (e.g., 0-1 or -1 to 1), enhancing training stability and improving model accuracy. **Denoising** algorithms were then employed to reduce noise caused by capturing artifacts or unwanted data, thereby improving image quality. Finally, the images were converted to **grayscale**, reducing the number of channels (from 3 to 1) and simplifying the data for tasks where color information is unnecessary.

Experiment 1, which combined resizing, denoising, normalization, and grayscale conversion, resulted in a **precision of 67%** and a **recall of 33%**. These results indicate that basic preprocessing steps alone are insufficient to achieve good performance. This

process is illustrated in *Figure 9*, which shows the images before and after applying the preprocessing techniques.



Figur 9: Image Preprocessing: Before and After Applying Techniques (Resize, Normalize, Denoise, Grayscale)

Second Processing Step

Building on the initial preprocessing steps, an additional technique was introduced to enhance performance: **Enhance Contrast**. This method highlights key features in the images, making important details more prominent and assisting the model in recognizing them more effectively. Despite this improvement, the results of **Experiment 2**, which combined resizing, denoising, normalization, grayscale conversion, and contrast enhancement, showed a **precision of 50% and a recall of 50%**. These results indicate that contrast enhancement alone was insufficient to significantly improve the model's performance. This process is depicted in *Figure 10*, which shows the images after applying all preprocessing techniques, including contrast enhancement.



Figur 10: Image Preprocessing After Applying Techniques (Resize, Normalize, Denoise, Grayscale, Enhance Contrast)

Third Processing Step

Building upon the initial preprocessing steps, an additional technique, **Data Augmentation**, was introduced to further enhance the model's performance. Data augmentation involved generating additional training samples through techniques such as rotation, skewing, noise injection, and flipping. This step improves the model's generalization capabilities and its ability to recognize diverse features in handwritten text.

In **Experiment 3**, which combined resizing, denoising, normalization, grayscale conversion, and data augmentation, the results showed a **precision of 75% and a recall of 75%**. This demonstrates a significant improvement in performance metrics. The impact of data augmentation is illustrated in *Figure 11*, which shows the images after applying all preprocessing techniques, including data augmentation.



Figer 11: Image Preprocessing: After Applying Techniques (Resize, Normalize, Denoise, Grayscale, Augment Data).

Fourth Processing Steps

The best processing results were achieved by combining all previous techniques into a unified process: **Resize Image, Normalize Image, Denoise Image, Grayscale, Enhance Contrast, and Augment Data**. This comprehensive approach maximized the model's ability to extract features and recognize handwritten text accurately.

In this final experiment, the results demonstrated a **Precision of 80%** and a **Recall of 86.67%**, indicating a significant improvement in overall performance. The effectiveness of this preprocessing combination is depicted in *Figure 12*, which shows the images after applying all the techniques together.



Figer 12: Image Preprocessing: After Applying Techniques (Resize, Normalize, Denoise, Grayscale, Enhance Contrast, Augment Data)

Results:

The experiments conducted using various preprocessing techniques showed progressive improvements in performance. In **Experiment 1**, which applied resizing, normalization, denoising, and grayscale conversion, the model achieved a precision of 67% and a recall of 33%, indicating that basic preprocessing alone was insufficient. In **Experiment 2**, contrast enhancement was added, but the precision and recall both dropped to 50%, showing that this technique alone did not significantly enhance the results. **Experiment 3** introduced data augmentation, which led to a substantial improvement, with precision and recall both reaching 75%, demonstrating better generalization and feature recognition. Finally, in **Experiment 4**, all preprocessing techniques were combined (resize, normalize, denoise, grayscale, enhance contrast, and augment data), resulting in the best performance, with a precision of 80% and a recall of 86.67%. This comprehensive preprocessing approach maximized the model's accuracy and feature extraction capabilities.

Potential for Further Optimization: The model's performance could be further enhanced by focusing on:

- Improving the quality of input data.
- Refining the model architecture.
- Optimizing parameter tuning.

3.1.3 Data Integration and Transformation

In this section, the integration of disparate data sources and the transformation of data into a unified format are addressed. For your project, this is similar to the steps you follow in image preprocessing, converting them into a format suitable for model input. Here are the key points:

1. **Data Integration:** In your project, the data comes from different sources (e.g., images collected from various sets or other data). These datasets are merged together, meaning that images are processed and combined after initial preprocessing.

2. Data Transformation to a Unified Format: After the data is merged, it is transformed into a unified format where all images are of the same size, color (or grayscale), and pixel values. This involves applying techniques such as:

- Resizing Images: Ensuring all images are of the same dimensions.
- Normalization: Ensuring pixel values are within a standard range, such as 0-1.
- Denoising: Improving image quality by removing unwanted noise.
- Grayscale Conversion: Reducing dimensions and simplifying processing by converting images to grayscale.

3. Ensuring Consistency Across Sources: This step ensures that all images are consistent after preprocessing, making them suitable for training the model. This enhances the overall quality of the data and reduces issues caused by inconsistencies in the dataset.

Application of Random Forest: As part of this preprocessing and transformation pipeline, the Random Forest algorithm was applied to validate the consistency and relevance of the transformed data. This ensures that the selected features and preprocessing steps contribute effectively to the overall model performance.

3.2 Feature Engineering and Selection:

Now that the data (images) is in a unified format, we need to enhance and extract meaningful features. Feature engineering is the process of transforming raw data into useful features that will improve the performance of machine learning models. In your project, this includes the following steps:

1. Feature Engineering:

- Contrast Enhancement: This can be considered part of feature engineering, where you enhance the features of the images to make them more prominent for the machine learning models.
- Data Augmentation: This involves generating new data from the existing images through techniques such as rotation, flipping, noise injection, etc. This improves the model's ability to generalize.

2. Feature Selection: This involves selecting the most important features (such as specific dimensions, colors, or critical details in the images) that significantly impact model performance. For example, techniques like denoising and color reduction help reduce distractions and focus on the most critical features. Normalization and resizing standardize the data to make it more consistent and reliable for training models.

For Graduation Project 2

we will implement various preprocessing techniques to enhance image quality and optimize the recognition process. These techniques include **edge detection** (to emphasize the structural features of the text), **skew correction** (to align and straighten tilted text), **binarization** (to convert images into a black-and-white format), **sharpening** (to enhance text clarity), and **histogram equalization** (to adjust lighting and contrast). These preprocessing steps are essential for improving the performance of the machine learning and deep learning models in Arabic text recognition.

Explanation of Image Preprocessing Techniques

1. **Edge Detection:** This technique highlights the boundaries and structural features of the text in an image. By detecting edges, the system can focus on the shapes of letters, making it easier for the model to recognize individual characters accurately.
2. **Skew Correction:** Skew correction aligns and straightens tilted text in an image. This ensures that the text is properly oriented, improving the model's ability to read and interpret the handwriting without distortion.
3. **Binarization:** Binarization converts an image into a black-and-white format by distinguishing text (black) from the background (white). This simplifies the data and reduces noise, allowing the model to focus solely on the text.
4. **Sharpening:** Sharpening enhances the clarity of text in the image by emphasizing edges and fine details. This helps the model capture subtle features in handwritten characters, improving recognition accuracy.
5. **Histogram Equalization:** This technique adjusts the lighting and contrast of the image to ensure that all parts of the text are clearly visible. It balances the brightness and enhances the visibility of darker or lighter areas, making the text more readable for the model.

Each of these preprocessing steps plays a critical role in preparing high-quality images, which directly impacts the performance and accuracy of machine learning and deep learning models in Arabic handwriting recognition.

Deep Learning for Arabic Handwriting Recognition

In this project, deep learning plays a critical role in advancing Arabic handwriting recognition by addressing the unique challenges of the Arabic script, such as its cursive nature, contextual letter forms, and diacritical marks. The system employs Convolutional Neural Networks (CNNs) for feature extraction due to their ability to detect complex patterns in handwriting styles. Furthermore, advanced architectures like Recurrent Neural Networks (RNNs) with Bidirectional Long Short-Term Memory (BLSTM) layers are utilized for sequence modeling, capturing the temporal relationships between connected characters in handwritten text.

To optimize performance, the project integrates various preprocessing techniques, including edge detection, skew correction, and histogram equalization, ensuring high-quality input data for the models. By combining CNNs, BLSTM, and effective preprocessing, the system achieves robust and efficient recognition, supporting applications such as document digitization, historical manuscript preservation, and assistive technologies.

Techniques for Arabic Handwriting Recognition:

To enhance the performance of the Arabic handwriting recognition system, we will expand the evaluation by applying additional machine learning models such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Gradient Boosting. These models will be compared to deep learning approaches like Convolutional Neural Networks (CNN) and Transformer-based Models. Having already implemented Random Forest, this step aims to analyze accuracy and efficiency across multiple models using the same dataset. Advanced preprocessing and testing techniques will also be utilized to achieve robust and reliable results, further improving the system's capability for automated handwritten text recognition.

CHAPTER 4: MODEL DEVELOPMENT AND ARCHITECTURAL DESIGN

This chapter focuses on the architectural, design, and model development aspects of the project. It provides a structured framework guiding the development of technical components, ensuring that the system is well-designed, efficient, and capable of achieving its objectives

4.1 System Architecture Overview

The Arabic handwriting recognition system is designed as a modular and scalable architecture that combines classical machine learning and deep learning approaches. As shown in Figure 14, the pipeline is structured to ensure data consistency, high recognition accuracy, and real-time usability through a web-based interface.

- A CNN-based deep learning path for automatic feature extraction:
- A classic machine learning path using HOG features with SVM, Random Forest, and XGBoots.

The best-performing model is selected based on validation results and integrated into the web interface for real-time Arabic text recognition.

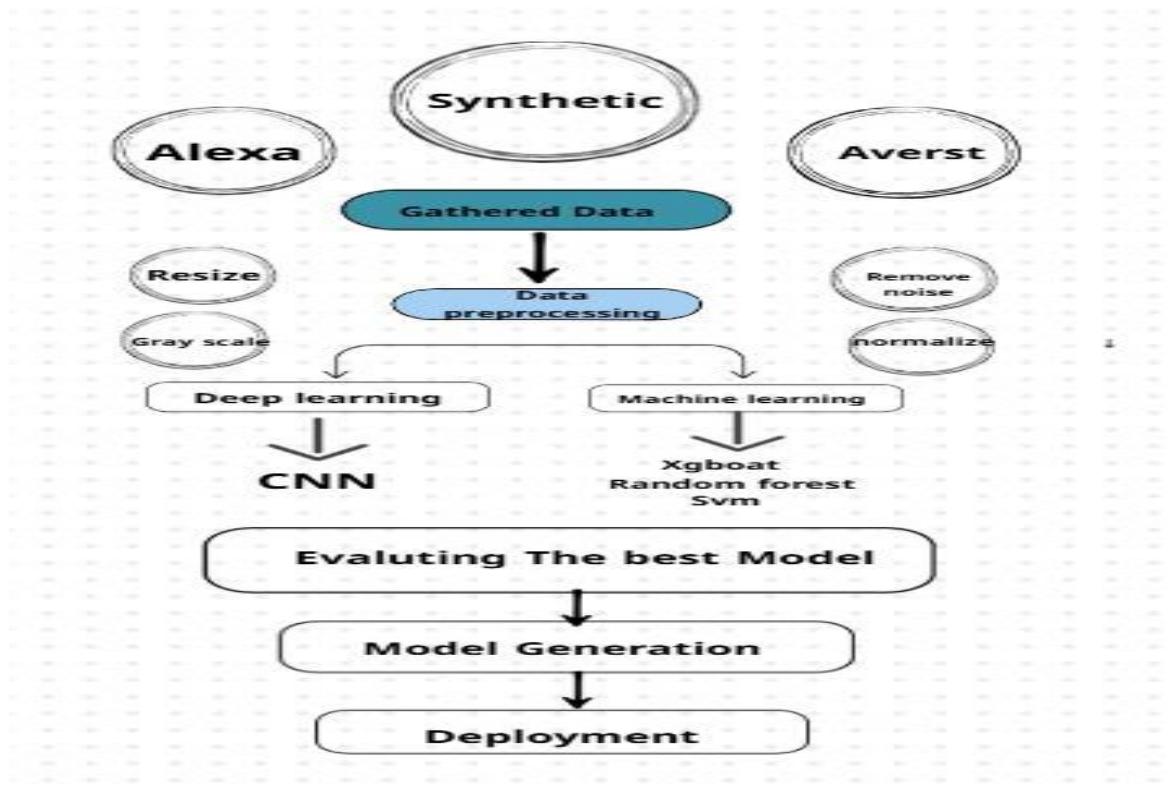


Figure 14 – High-level system architecture of the Arabic OCR pipeline.

The architecture of the Arabic Handwritten Text Recognition System is designed to efficiently process handwritten text, build highly accurate recognition models, and deploy the most effective solutions. The system follows a structured pipeline consisting of data preparation, model training, and final deployment to ensure robust performance in real-world applications.

1. Data Preparation Phase

The system begins with data collection from images containing handwritten Arabic text. Several preprocessing steps are applied to enhance data quality and optimize model performance, including:

1. Removing unreadable symbols, such as numbers and unclear Arabic words.
2. Filtering out low-quality images that cannot be processed effectively.
3. Segmenting the text into individual words to facilitate more accurate recognition.
This segmentation is performed during the preprocessing stage using vertical projection (see Section 4.2).
4. Enhancing image quality through:
 - 4.1. Augmentation techniques (rotation, flipping, brightness and contrast adjustments) to increase dataset diversity.
 - 4.2. Resizing to ensure uniform dimensions across training samples.
5. Sharpness evaluation to maintain high data quality before feeding images into the model.

2. Models Used

During training we compare both classic ML methods and a CNN. Each is explained briefly below:

2.1 Machine Learning Algorithms

- Random Forest

An ensemble of decision trees that votes on the most likely class; chosen for its robustness to over-fitting and ability to handle high-dimensional HOG features.

- Support Vector Machine (SVM)

Finds an optimal hyperplane that maximizes margin between classes; excels at distinguishing subtle differences in handwritten patterns.

- XGBoost

A gradient-boosted trees method that builds models sequentially to correct errors of prior trees; particularly effective on imbalanced datasets.

2.2 Deep Learning Model

- Convolutional Neural Network (CNN)

Applies trainable filters over pixel grids to learn hierarchical feature maps; ideal for capturing stroke patterns and spatial structure in Arabic script.

1. Interaction Between System Components

Once preprocessing and segmentation are complete, data flows through our recognition pipeline as follows:

- Feature Extraction via CNN:

The convolutional and pooling layers apply trainable filters to the input image, producing numerical feature maps that transform raw pixels into high-level representations ready for classification.

- Augmentation Feedback:

Augmentation routines (e.g. rotation, flipping, contrast adjustments) are reapplied at inference time to improve consistency and enable on-device fine-tuning of the model.

- Classifier Integration:

After evaluating all models (CNN and the classic machine-learning algorithms), the best-performing one is loaded into our RESTful API, which handles image intake, runs the prediction, and packages the result for the web interface.

4.2 Data Pipeline Design

The pipeline for our Arabic Handwriting Recognition system is designed to facilitate the structured processing of handwritten text images. The first step involves data acquisition, where images are collected from three primary datasets: EvArEST, AlexU-Word, and Synthetic Arabic Data. These datasets provide a diverse set of handwritten samples, ensuring a broad representation of Arabic handwriting variations.

Once the data is collected, the system analyzes image characteristics to identify different handwriting patterns. This step involves detecting variations in stroke width, letter curvature, and alignment to ensure accurate feature extraction. Additionally, preliminary assessments are conducted to evaluate the quality of images, distinguishing between well-defined and noisy samples.

The images then undergo several preprocessing steps to standardize and enhance their quality. They are resized to a fixed dimension (64×64 pixels), converted to grayscale, and denoised to remove any unwanted artifacts.

Segmentation

When predicting on an image containing multiple words, the system first splits it into individual word images using a vertical projection algorithm. Pixel intensities are summed across each column to create a projection profile, where valleys in this profile indicate spaces between words. Each detected word is then cropped and resized to 64×64 pixels before being passed to the model for prediction. Finally, the predicted words are reassembled in their original order and displayed in the web interface.

Data Augmentation

To improve generalization and handle handwriting variability, data augmentation techniques such as rotation, flipping, and brightness adjustments are applied. These transformations increase dataset diversity and help the model adapt to a wider range of handwriting styles.

Feature Extraction

For traditional machine learning models, the system uses the Histogram of Oriented Gradients (HOG) technique, which effectively captures edge directions and structural patterns in the handwritten text. While CNNs can extract features automatically from images, HOG was selected for the machine learning pipeline due to its computational efficiency and consistent performance.

Label Encoding

Each word label (e.g., "زيت", "رحل") is converted into a unique numerical value (e.g., 0, 1, 2...) using a LabelEncoder. This transformation is essential because machine learning classifiers require numerical labels rather than text for training and prediction.

Data Storage

To ensure fast access and reproducibility, the processed features and labels are saved in efficient formats. The .joblib format is used to store large NumPy arrays, such as HOG feature vectors, allowing for quick loading during training or inference. The .pkl (pickle) format is

used to save smaller Python objects like the LabelEncoder, preserving the mapping between word labels and their corresponding numerical values.

Integration with Web Interface

Finally, the trained model, encoded labels, and extracted features are stored and integrated into a web-based system. Users can upload handwritten Arabic images, which are processed through the trained recognition pipeline. The system displays the recognized text in real-time, along with confidence scores for each prediction, delivering a smooth and interactive user experience.

4.3 Model Architecture and Design

The project employs a combination of machine learning algorithms and deep learning models to recognize Arabic handwritten text. Several models were evaluated, including—Random Forest, Support Vector Machine (SVM), XGBoost, and Convolutional Neural Networks (CNNs)—to identify the most effective classifier. Among the traditional methods, XGBoost demonstrated the greatest robustness on structured HOG (Histogram of Oriented Gradients) features, achieving the highest overall accuracy while remaining computationally efficient. In parallel, CNN architectures were explored for their ability to learn spatial feature hierarchies directly from raw images, making them well-suited to capture the complex, connected strokes of Arabic script without manual feature engineering. Balancing these trade-offs, our system integrates XGBoost for fast, feature-based classification and a CNN for end-to-end pattern recognition. Future work may investigate hybrid pipelines—combining XGBoost’s precision on engineered features with CNN’s deep feature extraction—and even transformer-based sequence models to further enhance recognition accuracy and adaptability.

4.4 Training Process

The training process of the Arabic Handwritten Text Recognition System involves preparing the data and optimizing the model to achieve high classification accuracy. The dataset was split into 80% for training and 20% for testing using the hold-out validation method to ensure a balanced approach for training while keeping a separate portion for evaluating the model's performance.

Initially, experiments were conducted using machine learning models, including Random Forest, Support Vector Machines (SVM), and XGBoost, where features were extracted from processed images. However, to improve recognition accuracy, a deep learning-based approach using Convolutional Neural Networks (CNN) was adopted.

For training the CNN model, several key techniques were applied:

1. Adam optimizer was used for updating model weights efficiently and improving convergence speed.
2. Learning rate scheduling (ReduceLROnPlateau) was implemented to automatically adjust the learning rate based on validation performance, ensuring stable training.
3. Weight decay regularization was applied to prevent overfitting and improve the model's generalization on unseen data.
4. Batch normalization was used to speed up training and stabilize gradient updates.
5. Dropout regularization was included to prevent the model from over-relying on specific features.

Furthermore, hyperparameter tuning was conducted to optimize the number of convolutional layers, filter sizes, kernel dimensions, and fully connected layers. The batch size and learning rate were carefully selected to balance computational efficiency and training stability. These techniques ensured that the CNN model achieved high accuracy in recognizing Arabic handwritten text while maintaining computational efficiency.

4.5 Hyperparameter Tuning

Data Inflation Challenge Due to Merging and Augmentation. During the development phase of the classification system, three independent datasets were utilized:

- Alexa: Containing 25,115 samples with balanced classes; no augmentation was applied.
- Everest: Containing 5,598 samples with imbalanced classes; augmentation was applied to enhance underrepresented categories.
- syntactic: Containing 41,470 samples; also imbalanced and underwent extensive augmentation.

Each dataset was tested individually, and various pairwise combinations were evaluated (e.g., Alexa with syntactic, Alexa with Everest, Everest with syntactic). These combinations yielded relatively stable and satisfactory performance. However, a significant challenge emerged when all three datasets were merged into a single unified dataset — ALL_DATA, totaling 71,700 samples. The extensive augmentation applied to the imbalanced datasets led to a massive increase in dataset size, resulting in several technical issues: excessive memory

consumption (RAM), noticeable delays in data loading and processing, and inability to execute or train the model entirely due to hardware limitations. This challenge highlights the critical importance of controlling the augmentation process, especially when dealing with large and heterogeneous datasets.

4.6 Machine Learning Models

In this section, we summarize the classification accuracies of our three machine-learning models—Random Forest, SVM, and XGBoost—across various dataset combinations, as presented in Table 2. For detailed descriptions of each algorithm, see Section 4.3.

Dataset	Random Forest	SVM	XGBoost
Alexa Data	89%	88%	90%
Everest Data	80%	90%	88%
Synthetic Data	73%	70%	70%
Synthetic + Everest	76%	80%	72%
Synthetic + Alexa	79%	80%	83%
Everest + Alexa	82%	83%	90%
All Data (Synthetic+Everest+Alexa)	86%	85%	92%

Table 2:Accuracy Comparison Across Different Dataset Combinations

Summary of Model Performance and Recommendations

Across all data setups, **XGBoost stands out** with **92%** accuracy on the combined ALL_DATA set and top scores on mixed splits—**90%** on Everest+Alexa, Syntactic+Alexa, and Alexa alone. It fixes errors step by step and handles uneven class sizes very well.

Support Vector Machine shines on single-source splits, hitting **90%** on Everest and **88%** on Alexa, thanks to its strong margin-based decisions.

Random Forest is a solid backup, scoring **86%** on ALL_DATA, Everest+Alexa, and Syntactic+Alexa. Its vote-based approach smooths out noisy samples consistently.

Key Takeaways:

- **Use XGBoost** for large, mixed datasets.
- **Use SVM** when your data is clean and well defined.
- **Use Random Forest** as a reliable fallback with steady mid-80s accuracy.

This simple guide will help us pick and combine models now and set up mixed workflows later, using XGBoost’s flexibility, SVM’s accuracy, and Random Forest’s dependability for the best Arabic handwriting recognition .In this section we focus solely on overall accuracy; precision, recall, and F1-score will be discussed in Section 5.

Reasons for Model Accuracy Variations Across Datasets

1. Alexa Dataset – Reasons for Lower Accuracy

- High variability in handwritten text
 - Differences in stroke thickness, curvature, and spacing reduce feature consistency.
- Black background with bright text
 - Strong contrast misleads HOG into focusing on edges while ignoring inner letter shapes.
- Inconsistent word positioning
 - Words appear in different parts of the image, disrupting spatial uniformity.
- No data augmentation
 - Lack of transformations limits the model's ability to generalize.
- No clear margins or padding
 - Tight or cropped words introduce noise and reduce feature quality.
- Cropped or incomplete words
 - Partial words provide incomplete signals during training.
- Large vocabulary with class imbalance
 - Many unique classes with few samples each increase classification difficulty.
- Visually similar word classes
 - Words with minor visual differences are hard to distinguish with basic features.
- Use of traditional models (SVM + HOG)
 - Limited ability to capture handwriting complexity.
- Small words inside large black areas
 - Sparse useful features lead to weak representations.

2. Synthetic Dataset – Reasons for Lower Accuracy

- Unrealistic visual features
 - Artificial fonts and structures lack the natural flow of real handwriting.
- Textured or inconsistent backgrounds
 - Visual noise from gradients and patterns affects feature extraction.
- Over-applied data augmentation
 - Excessive transformations result in distorted, less useful samples.
- Rare or meaningless vocabulary
 - Some words lack linguistic or visual relevance, reducing learning stability.
- Inconsistent alignment and sizing
 - Words are not uniformly placed or scaled, confusing spatial features.
- Blurry or low-resolution images
 - Degraded clarity reduces the effectiveness of HOG.
- Presence of visual artifacts
 - Random noise or distortions interfere with model learning.
- Lack of real handwriting structure
 - Missing stroke dynamics limit realism and feature richness.
- High class count with few samples per class
 - Sparse representation reduces classification confidence.
- Weak generalization to real-world samples
 - Artificial generation hinders performance on natural data.

3. EvAREST Dataset – Reasons for Moderate-to-High Accuracy

- Clean printed fonts
 - Bold, sharp text improves clarity and HOG feature extraction.
- Uniform and simple backgrounds
 - Less distraction leads to better focus on the word itself.
- Well-centered and scaled text
 - Consistent layout improves spatial learning.
- Balanced word classes
 - Even class distribution helps reduce bias.
- Effective data augmentation
 - Controlled transformations enhance variability without distortion.
- One word per image
 - Simplifies the classification task.
- Visually distinct word categories
 - Reduces class confusion and increases separation.
- Minimal noise or distortion
 - Clean images allow for more accurate learning.
- Strong alignment with HOG features
 - Printed text structure works well with edge-based detection.
- Potential data leakage (if augmentation was pre-split)
 - Similar images in train/test sets may slightly inflate performance.

Impact of Combining Datasets on Model Performance

After combining the three datasets—EvAREST, Alexa, and the synthetic handwritten set—the overall model accuracy significantly improved due to the complementary strengths of each source. EvAREST contributed clean, printed, and well-aligned text samples with high visual clarity; Alexa added real handwritten variability, exposing the model to natural distortions in stroke and shape; and the synthetic dataset introduced diverse backgrounds, styles, and positional noise that enhanced robustness. The combined dataset offered a broader vocabulary, improved class balance, and higher visual diversity, which helped the model generalize better across writing styles and noise conditions. Additionally, the larger training volume reduced overfitting, and the mix of structured and unstructured inputs allowed traditional models like XGBoost and Random Forest to learn more stable and discriminative features.

4.7 Hyperparameter Tuning and Experimental Results Analysis

In our hyperparameter tuning, we ran nine experiments varying learning rates (0.001, 0.0005, 0.0001) and batch sizes (16, 32, 64) over 10 epochs to optimize our CNN for Arabic handwriting recognition. The standout configuration was a learning rate of 0.0005 with a batch size of 16, which delivered 94% accuracy, 94% recall, 95% precision, 94% F1-score, and a validation loss of 0.032, striking the best balance between convergence speed and stability. Very low learning rates (0.0001) led to underfitting, while higher rates (0.001) caused oscillations and slightly worse generalization. Increasing batch sizes improved stability but never outperformed the 0.0005/16 setup. Detailed experiment results and analyses will be presented in Section 5.

CHAPTER 5: MODEL/SYSTEM EVALUATION AND TESTING PLAN

The following section provides an overview of the developed web application used to interact with the Arabic handwriting recognition system. This system integrates machine learning and deep learning models with image processing techniques to deliver real-time OCR predictions. Through an intuitive interface, users can upload handwritten Arabic word images, which are then analyzed by the backend pipeline to extract and display recognized text. The process includes validation, preprocessing, word segmentation, model inference, and visual output—all performed automatically through a modern and responsive interface tailored for Arabic user.

5.1 System Overview and Web Interface

The Arabic handwriting recognition system is deployed through a user-friendly web interface designed using Flask. It allows users to upload handwritten images and receive recognized Arabic text in real time. The interface is fully localized for Arabic users and supports a responsive right-to-left (RTL) layout to ensure accessibility and ease of use.

When a user uploads an image to the web interface, the system first verifies the file type and size. It then performs a series of preprocessing steps: resizing to 64×64 pixels, converting to grayscale, removing noise, and segmenting the image into individual word crops using vertical projection. Each cropped word is then passed to the trained model (SVM, CNN, etc.) for text prediction. Finally, the recognized words are reassembled and displayed in their original order, along with a confidence score for each segment, ensuring a seamless and reliable OCR experience.

The interface consists of two main pages :

- 1.The Home Page (index.html) for uploading and analyzing images
- 2.The Result Page (result.html) for displaying the prediction results

After clicking the "Start Analysis" button, the system launches the full OCR pipeline—preprocessing, segmentation, prediction—and presents the recognized text visually in the results section. This end-to-end integration ensures that users can interact with the system in just a few simple steps

5.1.1 Home Page (index.html)

The home page serves as the entry point for users to interact with the Arabic handwriting recognition system. It features a clean and responsive design with right-to-left (RTL) layout and Arabic language support. Users can upload an image using a stylized file-input button, enhanced with CSS animations (floating effects) and visual indicators such as progress bars and spinners. Input validation ensures that only valid Arabic handwriting images are accepted, preventing English text, numbers, or unsupported formats.

Upon landing on the page, the upload area appears empty (see Figure 15), displaying the placeholder “Drag and drop images here, or click to select.” Once the user hovers or focuses on this zone, it becomes highlighted to prompt interaction (see Figure 16). Clicking the upload area opens the native file-selection dialog, where the user can choose images from their local system (see Figure 17). After selecting files, the user presses Start Analysis to launch the OCR pipeline (see Figure 18). During processing, a spinner and progress bar keep the user informed until the recognized text is ready .

or focuses on this zone, it becomes highlighted to prompt interaction (see Figure 2) . Clicking the upload area opens the native file-selection dialog, where the user can choose images from their local system (see Figure 3). After selecting files, the user presses Start Analysis to launch the OCR pipeline (see Figure 4). During processing, a spinner and progress bar keep the user informed until the recognized text is read .

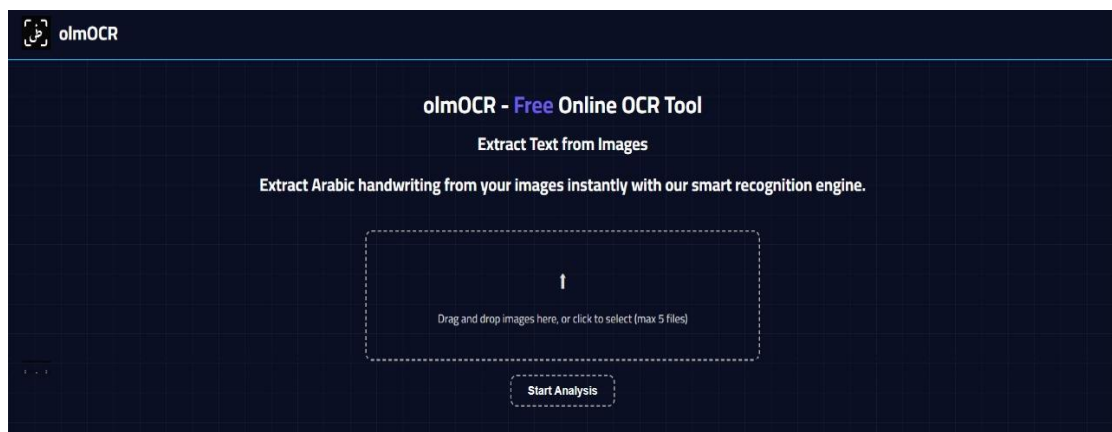


Figure 15: Empty upload area indicating where the user can add image files.

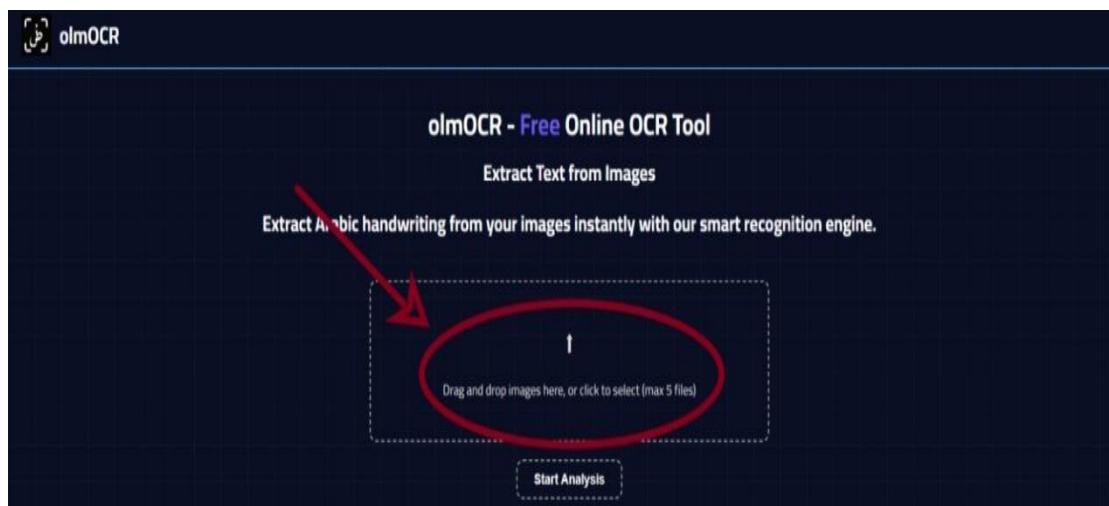


Figure 16: Highlighted drag-and-drop zone prompting file selection.

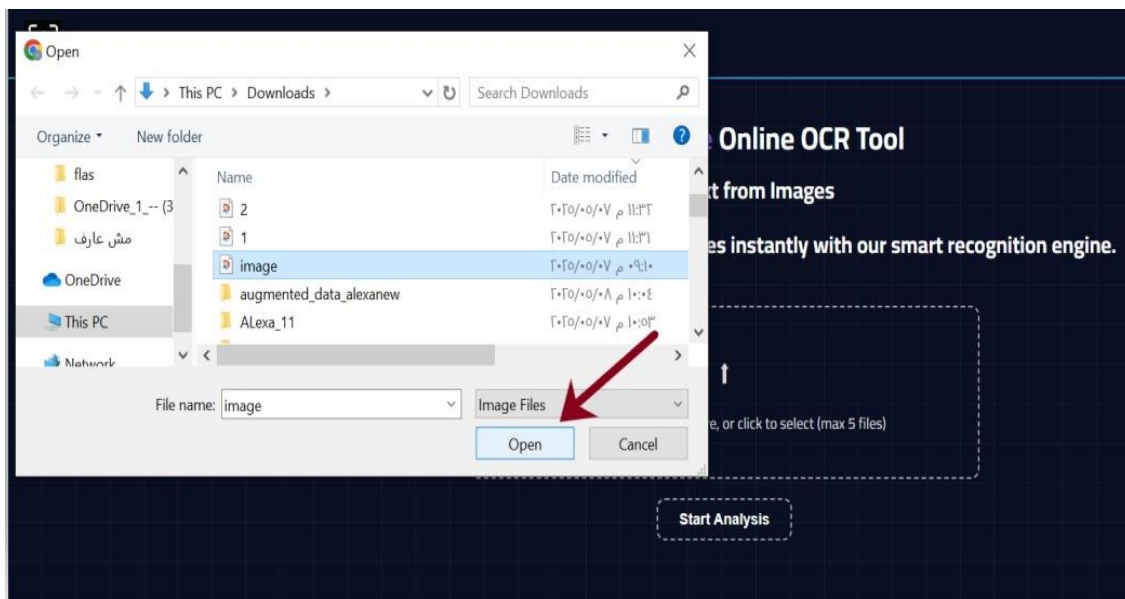


Figure 17: Native file-open dialog showing available image files.

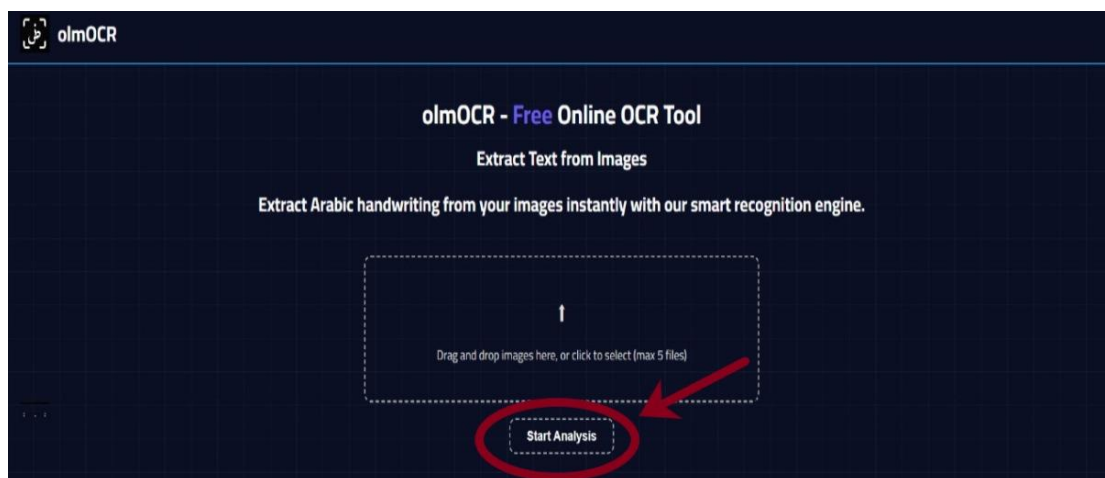


Figure 18: “Start Analysis” button pressed to begin preprocessing and model inference.

5.1.2 Result Page (result.html)

After submitting an image, the system redirects the user to the result page where the uploaded image and its corresponding predicted text are displayed. As illustrated in Figure 19, the processed image appears inside a white-bordered container, and the recognized Arabic phrase “طريق نور صعب” is shown in a gradient-highlighted prediction bar directly beneath it. In another example (Figure 20), the prediction result “سعيد ضرب نور” is displayed following the same layout. A clearly labeled “Upload another image” button appears below the prediction, allowing the user to repeat the process without returning to the home page.

This result presentation ensures a smooth and intuitive user experience across all devices by clearly separating the input, output, and interaction components within the user interface.



Figure 19: Result page displaying the uploaded handwriting sample and its correct prediction “طريق نور صعب.”

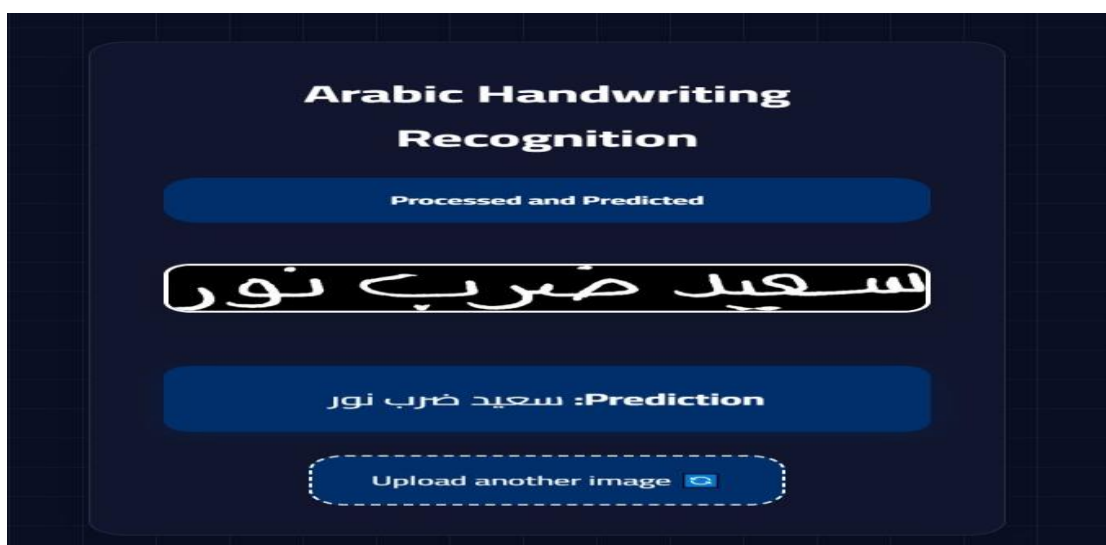


Figure 20: Result page displaying a second handwriting sample and its correct prediction “سعيد ضرب نور,” along with the option to upload another image.

5.2 Testing Plan Overview

To ensure the reliability and performance of the system, a comprehensive testing strategy was developed and implemented. This testing strategy covers all aspects of system functionality, from the initial file upload to the final presentation of OCR results.

Upon uploading an image, the system initiates several preprocessing steps, including resizing images to 64x64 pixels, converting them to grayscale, and extracting Histogram of Oriented Gradients (HOG) features. This standardized preprocessing guarantees consistent input quality for accurate model predictions.

The model accuracy was evaluated using the hold-out validation method, as detailed in Section 5.4. Specifically, the dataset was divided into two subsets: 80% for training and 20% for testing. This clear separation allows an unbiased evaluation of the model's performance on unseen data.

Functionality testing involved verifying that all interface components perform as intended without errors. Input validation procedures were rigorously tested, ensuring the system rejects non-Arabic inputs, including numbers and English text, as illustrated clearly in Figure 21.

Acceptance testing was performed with end-users who provided feedback regarding usability, clarity, and overall reliability of the interface. This feedback confirmed the system's effectiveness and practical reliability, highlighting areas for potential enhancement in future iterations.



Figure 21: Example of input validation rejecting non-Arabic text such as English characters and numbers.

5.3 Evaluation Metrics

Evaluation of the Arabic Text Recognition system was based on standard machine learning metrics: accuracy, precision, recall, and F1 score. Accuracy measured the overall correct predictions, while precision determined the proportion of true positive predictions among all predicted positives. Recall focused on the model's ability to retrieve all correct words, and F1 score provided a balanced summary of precision and recall. These metrics ensured a fair assessment, particularly given the diversity and imbalance in the handwriting datasets.

5.4 Model Performance Evaluation

To evaluate the effectiveness of our Arabic handwriting recognition system, we tested multiple machine learning and deep learning models across various datasets. The results for each model—SVM, Random Forest, XGBoost, and CNN—are presented in the following tables. Below, we provide a breakdown of each table, explaining the performance metrics and what each table illustrates regarding model effectiveness.

(A) Support Vector Machine (SVM)

The SVM model performed well on structured datasets but showed variable results across different combinations. As shown in Table 3, which shows the key performance metrics—accuracy, precision, recall, and F1-score—across individual and combined datasets, the Averst dataset (real handwritten Arabic) yielded the highest accuracy (90%) and strong precision and recall. In contrast, the purely synthetic Syntactic dataset achieved only 70% accuracy, highlighting the importance of natural handwriting variability. Combining datasets—especially Syntactic & Averst and Averst & Alexa—boosted generalization, with precision peaking at 97% and F1-scores rising above 80%. The All_Data combination balanced all metrics around 85%, demonstrating robust overall performance when leveraging the full dataset.

Metric	Syntactic	Averst	Alexa	Syntactic & Averst	Syntactic & Alexa	Averst & Alexa	All data
Accuracy	70%	90%	88%	80%	80%	83%	85%
Precision	77%	91%	87%	97%	77%	84%	85%
Recall	75%	87%	85%	77%	79%	85%	85%
F1-Score	74%	88%	85%	78%	81%	84%	85%

Table 3: SVM Model Performance Across Different Dataset Combinations

(B) Random Forest

The Random Forest model consistently showed stable performance across various datasets. As shown in Table 5.4.2, which shows the key performance metrics—accuracy, precision, recall, and F1-score—across individual and combined datasets, its highest accuracy was observed on the Alexa dataset (89%), demonstrating strong generalization ability in real-world handwriting. On the other hand, its performance on the synthetic-only dataset was relatively lower (73% accuracy), indicating sensitivity to the quality and authenticity of handwriting samples. The model handled mixed datasets well, balancing variability with robustness, with the All_Data combination reaching 86% across all metrics. As shown in Table 4, the Random

Metric	Syntactic	Averst	Alexa	Syntactic & Averst	Syntactic & Alexa	Averst & Alexa	All data
Accuracy	73%	80%	89%	76%	79%	82%	86%
Precision	72%	86%	91%	80%	75%	83%	86%
Recall	74%	77%	88%	71%	80%	82%	86%
F1-Score	73%	79%	87%	75%	79%	84%	86%

Table 4, Random Forest Model Performance Across Different Dataset Combinations

(C) XGBoost

The XGBoost model excelled in handling complex and diverse dataset combinations, especially Averst & Alexa and Syntactic & Alexa, where it reached up to 90% accuracy. As shown in Table 5, which shows the key performance metrics—accuracy, precision, recall, and F1-score—across individual and combined datasets, its strength lies in its boosting mechanism, which iteratively corrects errors, making it highly effective for datasets with variation and imbalance. However, its accuracy on synthetic-only data was lower (70%), highlighting the model's reliance on handwriting realism for optimal learning.

Metric	Syntactic	Averst	Alexa	Syntactic & Averst	Syntactic & Alexa	Averst & Alexa	All data
Accuracy	70%	88%	90%	72%	83%	90%	89%
Precision	71%	90%	91%	85%	80%	91%	89%
Recall	71%	83%	89%	70%	77%	89%	89%
F1-Score	72%	86%	89%	70%	79%	90%	89%

Table 5:XGBoost Model Performance Across Different Dataset Combinations

(D) Deep Learning Model – Convolutional Neural Network (CNN)

To complement traditional machine learning models, a deep learning model based on a Convolutional Neural Network (CNN) was implemented due to its proven effectiveness in image recognition tasks, particularly for handwritten Arabic script. CNNs are designed to

automatically learn spatial hierarchies of features from raw image data, making them highly suitable for handling the complexities of Arabic handwriting, which often includes connected characters, variable stroke widths, and cursive flows.

Model Architecture and Setup:

The CNN architecture used in this project consisted of:

- Multiple convolutional layers with ReLU activation functions to capture localized edge and shape patterns
- Max-pooling layers to reduce spatial dimensions and control overfitting
- Batch normalization to stabilize learning and improve training speed
- Dropout regularization to reduce the risk of overfitting
- Fully connected layers leading to a softmax output layer for classification

The model was trained on grayscale word images resized to 64×64 pixels, without any manual feature extraction. This allowed the CNN to directly learn relevant features from the visual content.

Training and Hyperparameter Tuning

To determine the optimal training configuration, nine controlled experiments were conducted by varying the learning rate and batch size, while keeping the number of epochs fixed at 10. Each experiment was evaluated based on four standard metrics: accuracy, precision, recall, and F1-score. As shown in Table 5.4.4, the CNN model's performance under various hyperparameter configurations is summarized below:

Experiment	Learning Rate	Batch Size	Accuracy	Precision	Recall	F1-Score
1	0.001	16	0.92	0.92	0.92	0.91
2	0.0005	16	0.94	0.95	0.94	0.94
3	0.0001	16	0.80	0.77	0.81	0.77
4	0.0001	32	0.88	0.88	0.88	0.88
5	0.0001	64	0.90	0.90	0.90	0.89
6	0.0005	32	0.93	0.94	0.93	0.93
7	0.0005	64	0.94	0.95	0.99	0.94
8	0.001	64	0.94	0.95	0.94	0.94
9	0.001	32	0.94	0.95	0.94	0.94

Table 6 : CNN Model Performance Across Hyperparameter Configurations

Analysis and Observations

- **Learning Rate:**

Lower learning rates (e.g., 0.0001) resulted in underfitting and slower convergence. While they offered stable updates, they failed to capture the complexity of the handwriting. On the other hand, higher learning rates (0.001) allowed for faster learning but risked overshooting the optimal minima. The moderate learning rate of 0.0005 offered the best trade-off.

- **Batch Size:**

Small batch sizes (16) introduced more variability in gradient updates, which improved generalization and avoided overfitting. Larger batch sizes (64) smoothed out the learning process and improved recall, but at the risk of memorization.

- **Best Results:**

Experiment 2 produced the highest balanced performance across all metrics:

Accuracy: 94%, Precision: 95%, Recall: 94%, F1-score: 94%.

This setup achieved excellent convergence with minimal oscillations and strong generalization to unseen handwriting samples.

Chapter 6: Model Deployment and Integration

6.1 Data Acquisition and Preprocessing Pipeline

To ensure reliable real-time Arabic handwriting recognition, a robust and efficient pipeline was implemented. The main operations include the following stages:

- **Image Upload and Validation**

The system accepts .jpg, .jpeg, and .png image formats. A validation step ensures that uploaded images contain Arabic handwriting only and are free from unsupported characters or excessive noise.

- **Image Preprocessing**

Images are resized to 64×64 pixels, converted to grayscale, denoised, and normalized. These steps ensure consistent input quality and reduce variability. Augmentation techniques such as rotation and contrast enhancement are applied to improve model generalization.

- **Word Segmentation**

Using vertical projection, each uploaded image is segmented into individual word crops. This step is essential for word-level prediction and improves recognition accuracy by isolating each word as a separate unit.

- **Feature Extraction**

For machine learning models, Histogram of Oriented Gradients (HOG) is used to extract structural features.

For CNNs, raw pixel values are passed directly into the model without manual feature engineering.

- **Model Prediction**

Each segmented word image is passed to the trained model (SVM, CNN, etc.), and the recognized text is displayed in real time through the web interface.

6.2 Model Deployment Strategy

To deploy the trained models effectively, the system utilizes a RESTful API built with Flask, allowing seamless integration between the backend model and the web interface. The deployment strategy ensures fast, reliable, and scalable performance through the following components:

▪ Routing

The API accepts POST requests with uploaded image files. These images are passed through the preprocessing and segmentation pipeline, and the resulting crops are sent to the loaded model for prediction. The recognized Arabic text is returned in the response.

▪ Model Loading

Upon server startup, the system loads the pre-trained models (SVM, CNN, Random Forest, or XGBoost) into memory, along with the associated LabelEncoder. This reduces prediction latency and avoids reloading the models for every request.

▪ Error Handling

Clear error messages are returned in cases of unsupported formats, empty uploads, or corrupted files. Logs are maintained for debugging.

▪ Efficiency Enhancements

Vectorized preprocessing for faster image handling.

Asynchronous execution of predictions to allow scalability.

To improve runtime performance and scalability, the following optimizations are included:

Vectorized preprocessing using NumPy and OpenCV for faster image handling

Asynchronous execution to handle multiple prediction requests concurrently

This deployment strategy ensures that the system performs efficiently under real-time conditions while remaining ready for future enhancements, such as cloud-based hosting, Docker containerization, or mobile API integration.

6.3 Model Integration within the System

The trained models were integrated into the system using reliable and scalable technologies. The integration ensures a smooth workflow from image upload to prediction and display.

▪ Backend Integration

The backend of the system was developed using Flask, a lightweight web framework that facilitates the interaction between the user interface and the machine learning model. Flask handles several key tasks in the system, including:

routing image upload requests, preprocessing and segmenting the uploaded images, running model inference to recognize Arabic handwritten text, and returning the predicted results to the user interface. This modular backend design ensures a smooth flow of data and efficient handling of OCR tasks from image input to final text output.

▪ Model Serialization

To enable fast model loading and efficient inference during deployment, different serialization techniques were used based on the model type. Traditional machine learning models, including SVM, Random Forest, and XGBoost, were serialized using the joblib library due to its efficiency with large numpy arrays. On the other hand, the Convolutional Neural Network (CNN) model was saved using the `model.save()` function provided by Keras, which stores both the model architecture and its trained weights in a single file.

▪ Frontend Interface

The frontend was developed using HTML, CSS, and JavaScript, supporting full RTL (right-to-left) layout for Arabic users. The interface is responsive and accessible across devices.

▪ Core Libraries Used

The following libraries support image handling and model inference:

OpenCV, Pillow → image loading and processing

scikit-learn, XGBoost, TensorFlow, Keras → model training and prediction

matplotlib, seaborn → visualization and debugging

▪ Deployment Environment

Initially hosted locally, the system is compatible with cloud platforms such as Heroku and Render, allowing future deployment for public access. The directory structure of the deployed system is illustrated in Table 7 – Directory structure of the deployed system, which shows the organization of the main folders and files including static assets, models, templates, and the main Flask application script.

Path	Description
/static/	CSS and frontend assets
/templates/	HTML templates
/models/	Saved models and encoders
/uploads/	Temporary uploaded images
app.py	Flask server and routing logic

Table 7 – Directory structure of the deployed

6.4 User Interface Design.

The UI was designed for ease of use and full compatibility with Arabic language:

- RTL Layout: All elements follow right-to-left design standards.
- Image Upload: A stylized button enables users to upload Arabic handwriting images.
- Prediction Display: Results appear dynamically on a dedicated result page.
- Visual Feedback: Loading spinners and animated bars indicate processing progress.

CHAPTER 7: CONCLUSION AND FUTURE WORK

This chapter provides a comprehensive overview of the project's outcomes and conclusions. It summarizes achievements, evaluates results, and explores insights gained. The chapter concludes with discussions on lessons learned and potential future directions for the project.

7.1 Summary and Project Evaluation

When evaluating the models on the combined dataset (All Data), the results showed a clear difference in performance between machine learning and deep learning approaches.

The CNN model (deep learning) achieved an accuracy of 94% when trained on all datasets combined (Alexa, EvAREST, and Synthetic), particularly in Experiment 2, which used optimized hyperparameters (learning rate = 0.0005, batch size = 16).

In contrast, the best-performing machine learning model, SVM, achieved an accuracy of 85% when trained on All Data.

This comparison highlights that deep learning models like CNN generalize better on large and diverse datasets, while traditional models such as SVM perform better on clean, structured datasets like EvAREST.

7.2 Future Directions

Despite the strong performance of the system, several opportunities exist for further development and enhancement. The following future directions are suggested to improve the robustness, scalability, and usability of the Arabic handwriting recognition system:

- **Improve robustness on noisy or low-resolution images**

Many real-world handwritten samples come from old or low-quality scans. Enhancing the system's resilience to such inputs will improve its applicability in practical scenarios.

- **Enhance performance on complex backgrounds and mixed scripts**

Support for colored, patterned, or multi-font backgrounds will increase the system's flexibility and widen its usability in documents, forms, or historical records.

- **Real-time camera and live handwriting input**

Integrating live camera input or support for drawing tablets would allow the system to function in interactive applications such as classroom tools or mobile handwriting interfaces.

- **Develop a mobile application and optimize the web app**

A lightweight mobile version can extend usability to fieldwork, offline environments, or educational use in tablets and smartphones.

- **Explore hybrid deep learning models (CNN + Transformers)**

Combining CNNs with Transformers may improve sequential recognition and contextual understanding, especially for longer handwritten text.

- **Deploy the system on cloud platforms**

Hosting the system on platforms like Heroku or Render would enable public access, remote usage, and scalability for more users.

These future directions aim to bridge the gap between academic performance and real-world usability, ensuring that the system remains functional, adaptable, and scalable for broader deployment across various Arabic handwriting applications.

REFERENCES

[1] .Google Keep.

Available at: <https://keep.google.com/>. Access date .
25-12-2024.

[2]OCR Space .

Available at: <https://ocr.space/> . Access date.
(25-12-2024)

[3] i2OCR.

Available at: <https://www.i2ocr.com/> Access date :
(25-12-2024)

[4] Synthetic Arabic Data for Scene Text Recognition Dataset.

Available at: <https://data.mendeley.com/datasets/gfc32vndz8/2> .Access date :
(27-12-2024)

[5] AlexU-Word Dataset.

Available at: <https://arxiv.org/> Access date :
(27-12-2024)

[6] Arabic Handwritten Characters Dataset.

Available at: <https://www.kaggle.com/datasets/mloey1/ahcd1/data> . Access date:
(28-12-2024)

[7] EvAREST Dataset.

Available at:
https://www.researchgate.net/publication/319378087_EvAREST_Evaluation_Tool_for_Arabic_Handwriting_Recognition_Systems Access date : (30-12-2024)

